

Ресурсы, View и xml верстка

Червяков Алексей



Отметься на портале!



ДЗ

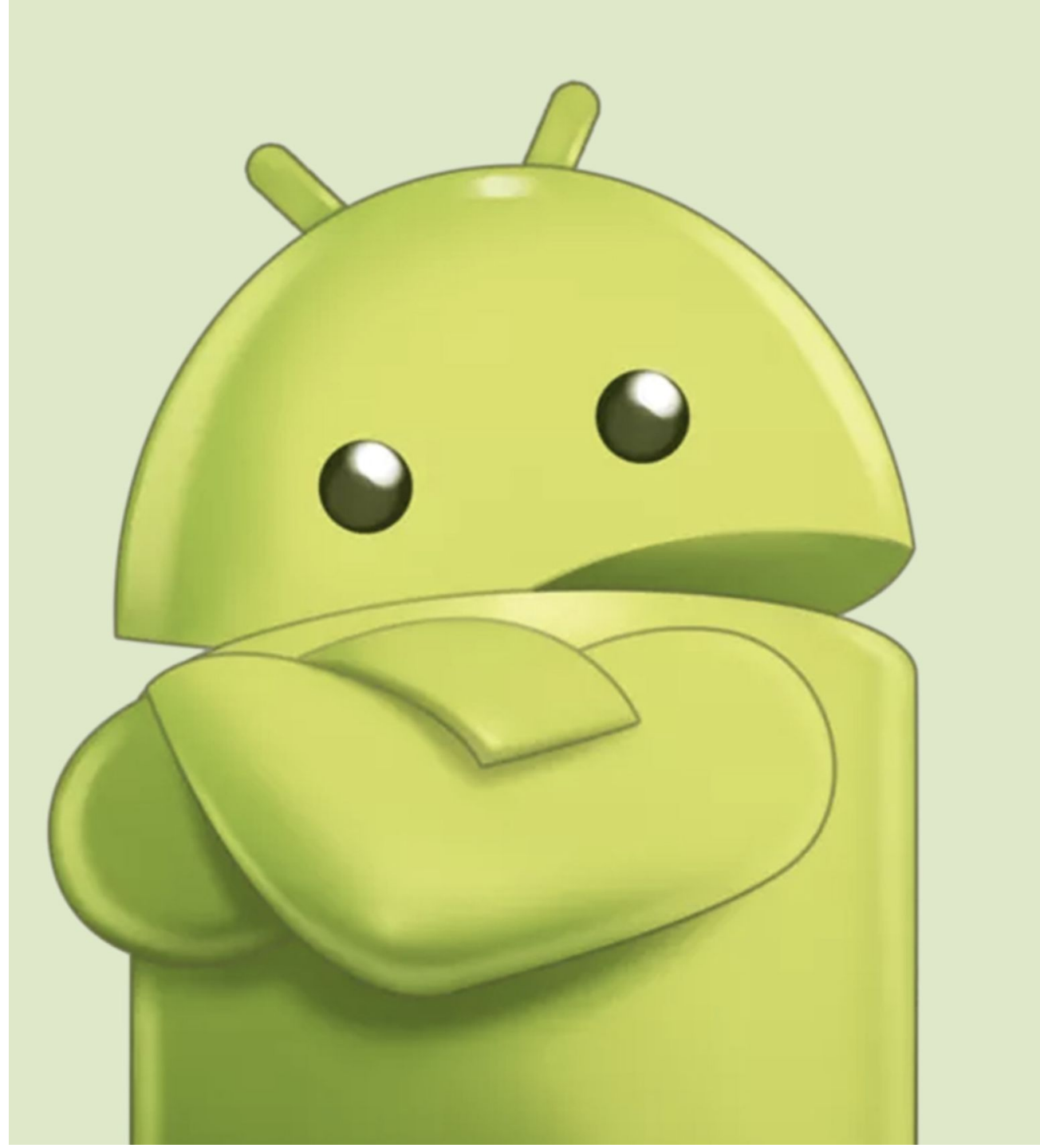


Тест для проверки усвоения материала по лекции.

- Сдать до 28.10.2025
- 15 минут на тест
- 5 баллов за задание

Краткий план

- Ресурсы
- XML
- View
- ViewGroup
- RecyclerView



Пользовательский интерфейс

Любое* приложение отображает пользователю графический интерфейс



Синтаксис

XML - язык разметки, определяющий теги (элементы) и их атрибуты.

Язык очень похож на HTML

Что такое ресурсы?

Что такое ресурсы?

Определение: Внешние элементы приложения

Хранение: Папка res со строгой структурой

Доступ: Через R класс

Преимущество: После изменения ресурса перекомпиляция только ресурсов

Типы ресурсов: Строки (strings.xml, plurals), Цвета (colors.xml), Размеры (dimens.xml), Массивы (arrays.xml), Drawable (изображения, shapes, векторы), Макеты (layout/), Меню (menu/), Анимации (anim/)

Адаптация под Устройство

Принцип: Разные версии ресурса для разных условий

Примеры квалификаторов: Язык (-ru), Ориентация (-land), Плотность экрана (-hdpi, -xhdpi), Версия API (-v21), Ночь/День (-night), Размер экрана

Механизм: Система выбирает наиболее подходящий ресурс из доступных. Если нет точного совпадения, используется дефолтный ресурс

Почему не px?

Проблема: Физические пиксели различаются на устройствах и мы получаем некорректное отображение

dp (Density-independent Pixels):

- Абстрактная единица на основе плотности экрана (dpi)
- **Формула:** $px = dp * (dpi / 160)$ (160 dpi - базовая плотность mdpi)
- **Где использовать:** Для размеров View (ширина, высота, отступы, поля)

sp (Scale-independent Pixels):

- Наследует логику dp
- Учитывает пользовательские настройки размера шрифта в системе
- **Где использовать:** Только для размеров текста (textSize)

Единообразие Внешнего Вида

- Стили (<style> в values/)
- Темы (<style> в values/)

Стили

Набор атрибутов для отдельных View

Применение: Атрибут `style="@style/MyStyle"` в XML макете

Темы

Набор атрибутов для всего приложения или отдельного экрана

Наследование: Атрибут parent, как в ООП

Применение: В манифесте <application android:theme>, <activity android:theme>

Базовые кирпичики интерфейса

Базовые кирпичики интерфейса

- View
- ViewGroup (наследник View)

View

Базовый класс для всех UI элементов

Отвечает: Отрисовка, Обработка касания, событий

Примеры: TextView, Button, ImageView

Обязательные атрибуты: android:layout_width, android:layout_height

ViewGroup

Контейнер для других View и ViewGroup

Отвечает: Размещение дочерних элементов

Примеры: LinearLayout, FrameLayout, ConstraintLayout, RecyclerView

Жизненный цикл View

onMeasure(): Определение своих размеров с учетом дочерних элементов

onLayout(): Определение позиций дочерних элементов

onDraw(): Непосредственная отрисовка на Canvas

Важно: Глубина иерархии View влияет на производительность

Детали

Атрибуты View:

- Общие: id, background, padding, margin, visibility
- Специфичные (TextView): text, textColor, textSize, textStyle

Пространства имен:

- android: Стандартные атрибуты Android SDK
- app: Кастомные атрибуты (из вашего кода, библиотек, AndroidX)
- tools: Атрибуты для превью в Android Studio (не попадают в APK)

Оптимизация

ViewStub:

- Ленивая загрузка макетов
- Легковесная невидимая заглушка в иерархии
- Инфлейтит целевой макет по требованию заменяя себя

Зачем: Ускорение первоначальной загрузки экрана, загрузка необязательных частей UI

Размещение Элементов

FrameLayout:

- Простейший контейнер
- Дочерние элементы “накладываются” друг на друга (Z-order)
- **Позиция:** По умолчанию (0,0)
- **Использовать:** Для перекрывающихся элементов, простых случаев, фрагментов

Размещение Элементов

LinearLayout:

- Элементы располагаются последовательно
- **Ориентация:** horizontal или vertical
- **Использовать:** Простые линейные списки элементов

Важно: Чем меньше вложенность ViewGroup, тем выше производительность

Мощный и Гибкий Макет

ConstraintLayout:

- Стандарт для сложных макетов
- **Позиционирование:** Относительно других элементов, родителя, направляющих, барьеров
- **Фи́чи:** Chains для группового поведения, отношения размеров (ratio)
- **Преимущества:** Снижение вложенности и повышение производительности, гибкость

Зачем нужен RecyclerView?

Проблемы “наивного” подхода (ScrollView + View):

- Создание всех View элементов списка сразу и огромное потребление памяти
- Замедление инициализации макета
- Прокрутка может быть не плавной
- Нет переиспользования

Решение: RecyclerView - компонент для эффективного отображения больших прокручиваемых списков данных

Как это работает?

RecyclerView: Сам контейнер-прокручиваемая область

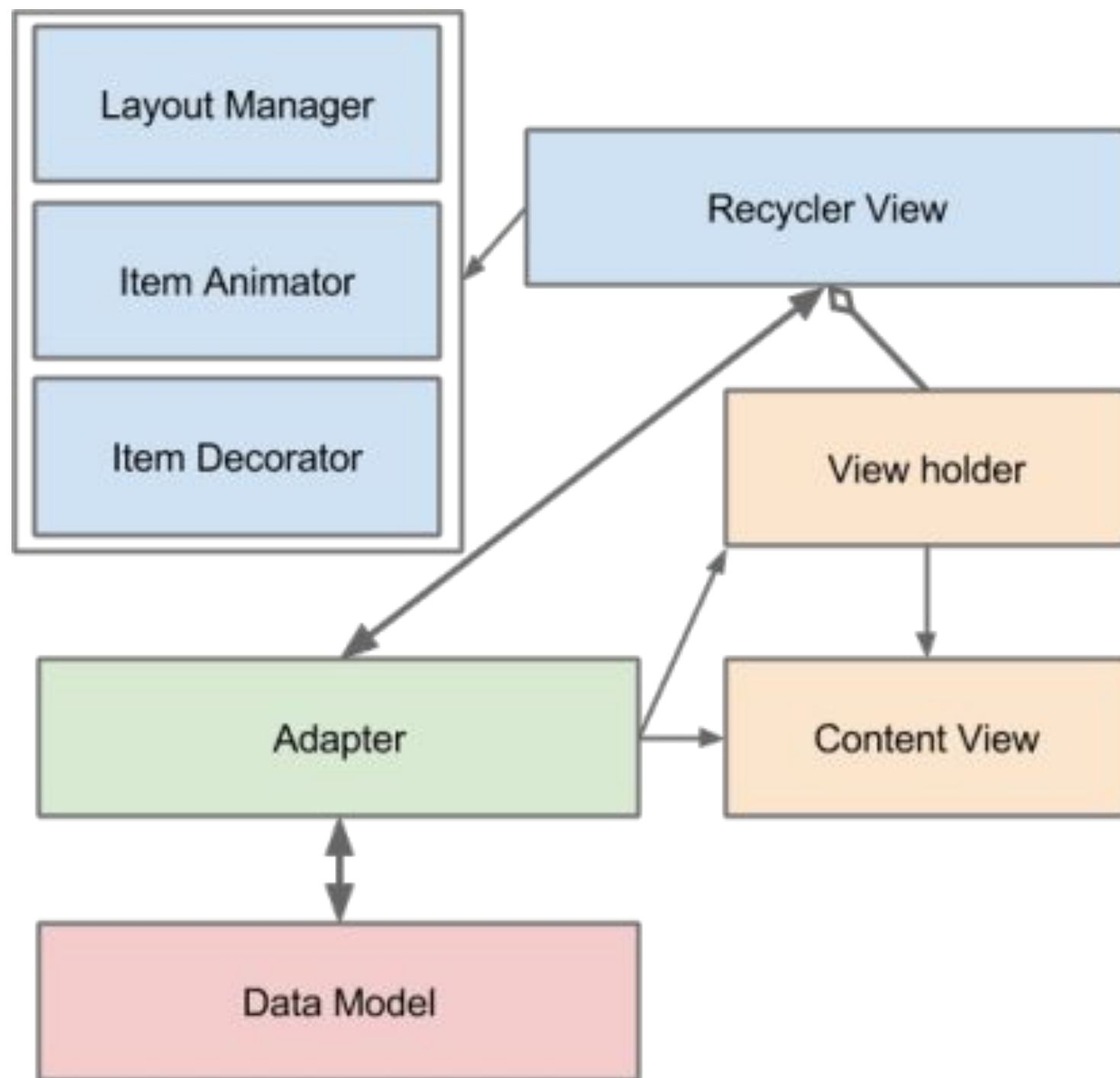
LayoutManager: Отвечает за компоновку элементов (линейно, сеткой, каскадом)

Adapter: “Мост” между данными и RecyclerView

ViewHolder: Кэширует ссылки на View внутри элемента списка

ItemDecoration: Рисует разделители, отступы, украшения вокруг элементов

ItemAnimator: Анимировывает добавление, удаление, изменение элементов



Принцип работы

LayoutManager определяет, какие элементы видны

RecyclerView запрашивает у **Adapter'a ViewHolder'ы** для этих позиций

Adapter ищет подходящий **ViewHolder** в пуле переиспользования

- Найден - передается в `onBindViewHolder()`
- Не найден - вызывается `onCreateViewHolder()` затем `onBindViewHolder()`

Подготовленный **ViewHolder** отображается

При прокрутке невидимые **ViewHolder** возвращаются в пул для переиспользования

Не только однообразные списки

Механизм:

- **Adapter.getItemViewType(position)** - возвращает целочисленный идентификатор типа макета для позиции
- **Adapter.onCreateViewHolder(parent, viewType)** - создает разные **ViewHolder** в зависимости от viewType и инфлейтит разные view
- **Adapter.onBindViewHolder(holder, position)** - привязывает данные к **ViewHolder** определенного типа

Пример: Заголовок секции (viewType = 0) + Обычный элемент (viewType = 1) + Реклама (viewType = 2)

Умные обновления и анимации

Проблема `notifyDataSetChanged()`

- Перерисовывает весь видимый список
- Сбрасывает анимации
- Неэффективно (перепривязка всех видимых ViewHolder)

Умные обновления и анимации

Решение: DiffUtil

- Вычисляет разницу между старым и новым списком
- Генерирует минимальный набор операций (добавить, удалить, переместить, изменить)
- Позволяет делать плавные анимации

Ключевые мысли

Ресурсы (res/) - внешние данные, управляемые через R

Использовать **dp** и **sp**

Стили/Темы - View vs Application

View/ViewGroup - базовые элементы

ConstraintLayout - мощный инструмент для сложных макетов

RecyclerView - стандарт для списков: Adapter, ViewHolder, LayoutManager

Обязательно: Переиспользование ViewHolder, DiffUtil для эффективных обновлений

Вопросы?

Спасибо
за внимание!

Червяков Алексей